ОТЧЕТ

по практической работе № 9

«Классы и объекты»

СОДЕРЖАНИЕ

1 Постановка задачи 2
1.1 Общая постановка задачи 2
1.2 Постановка задачи для конкретного варианта 3
2 Диаграммы классов 4
2.1 Часть 1 4
2.2 Часть 2 5
2.3 Часть 3 6
3 Код программы 7
3.1 Файл Time.cs 7
3.2 Файл TimeArray.cs 10
3.3 Файл UserInterface.cs 12
3.4 Файл Program.cs 14
4 Анализ покрытия кода тестами 18

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Общая постановка задачи

Часть 1.

1) Реализовать (в отдельном файле) определение нового класса/типа данных: закрытые атрибуты, свойства, конструкторы, инициализация и вывод атрибутов.
2) Для демонстрации работы с объектами написать главную функцию, в которой создаются объекты класса и выводится информация, которая содержится в атрибутах.
3) Написать функцию, реализующую указанное в варианте действие. Рассмотреть два варианта: статическую функцию и метод класса. В основной функции продемонстрировать работу функции.
4) Используя статическую компоненту класса, подсчитать количество созданных в программе объектов.

Часть 2.

1) Добавить к реализованному классу указанные в варианте перегруженные операции.
2) Написать демонстрационную программу, в которой создаются объекты пользовательских классов и выполняются указанные операции.

Часть 3.

1) Реализовать класс-коллекцию (в отдельном файле), полем которого является одномерный массив (без использования стандартных коллекций C#) из элементов заданного в варианте типа. В классе реализовать: конструктор без параметров; конструктор с параметрами, заполняющий элементы случайными значениями; конструктор с параметрами, позволяющий заполнить массив элементами, заданными пользователем с клавиатуры; метод для просмотра элементов массива.
2) Реализовать индексатор для доступа к элементам коллекции. Предусмотреть проверку выхода индекса за пределы массива.
3) Написать демонстрационную программу, позволяющую создать массив разными способами и распечатать элементы массива.

Подсчитать количество созданных объектов.

4) Написать функцию поиска минимального значения в классе `Program` для выполнения указанного в варианте задания (использовать индексатор и, если необходимо, перегрузить нужные для выполнения задачи операции).

1.2 Постановка задачи для конкретного варианта

Класс: `Time`.

Поля: `int hours` (часы), `int minutes` (минуты).

Часть 1. Метод — вычитание минут (`int`) из объекта типа `Time`. Необходимо учесть, что количество минут в объекте не может быть больше 59 и меньше 0. Результат должен быть типа `Time`. Функция реализуется в двух вариантах: как статическая функция и как метод класса.

Часть 2. Перегружаемые операции:

– унарные операции:
  – `-` — обнуление часов и минут в объекте типа `Time`;
  – `--` — вычитание минут из объекта типа `Time` (учесть, что минут и часов не может быть меньше 0);
– операции приведения типа:
  – `int` (неявная) — результатом является количество часов (минуты отбрасываются);
  – `bool` (явная) — результатом является `true`, если часы и минуты не равны нулю, и `false` в противном случае;
– бинарные операции:
  – `==` (`Time t1, Time t2`) — результатом является `true`, если значения часов и минут левого операнда равны соответствующим значениям правого операнда, и `false` — в противном случае;
  – `!=` (`Time t1, Time t2`) — результатом является `true`, если значения часов и минут левого операнда не равны соответствующим значениям правого операнда, и `false` — в противном случае.

Часть 3. Реализовать класс-коллекцию `TimeArray`, полем которого является одномерный массив `Time[] arr` элементов типа `Time`. В классе

`Program` написать функцию поиска минимального элемента массива (с использованием индексатора и перегруженной операции сравнения <).

2 ДИАГРАММЫ КЛАССОВ

На рисунках 1–3 приведены диаграммы классов для каждой части работы. Символ «+» обозначает открытый член класса, «-» — закрытый; статические члены подчёркнуты.

2.1 Часть 1

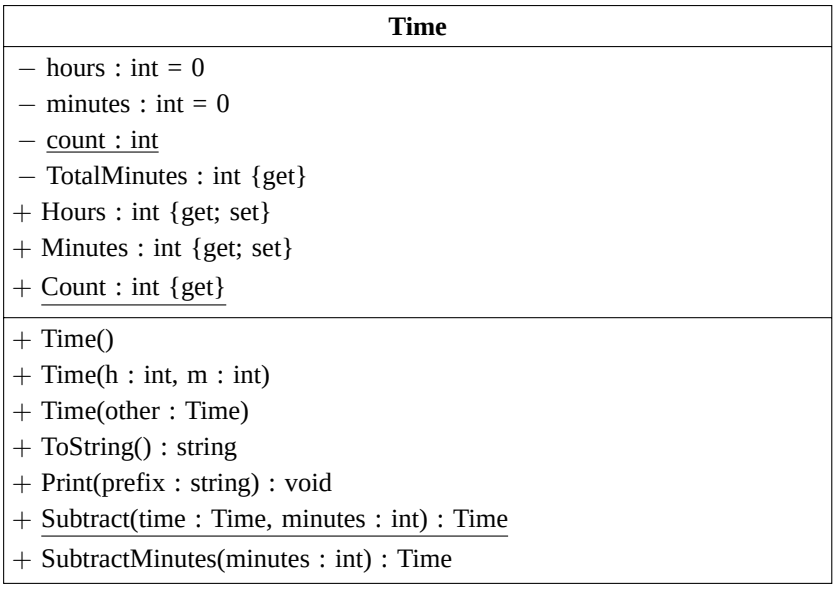


Рисунок 1 – Диаграмма классов. Часть 1

2.2 Часть 2

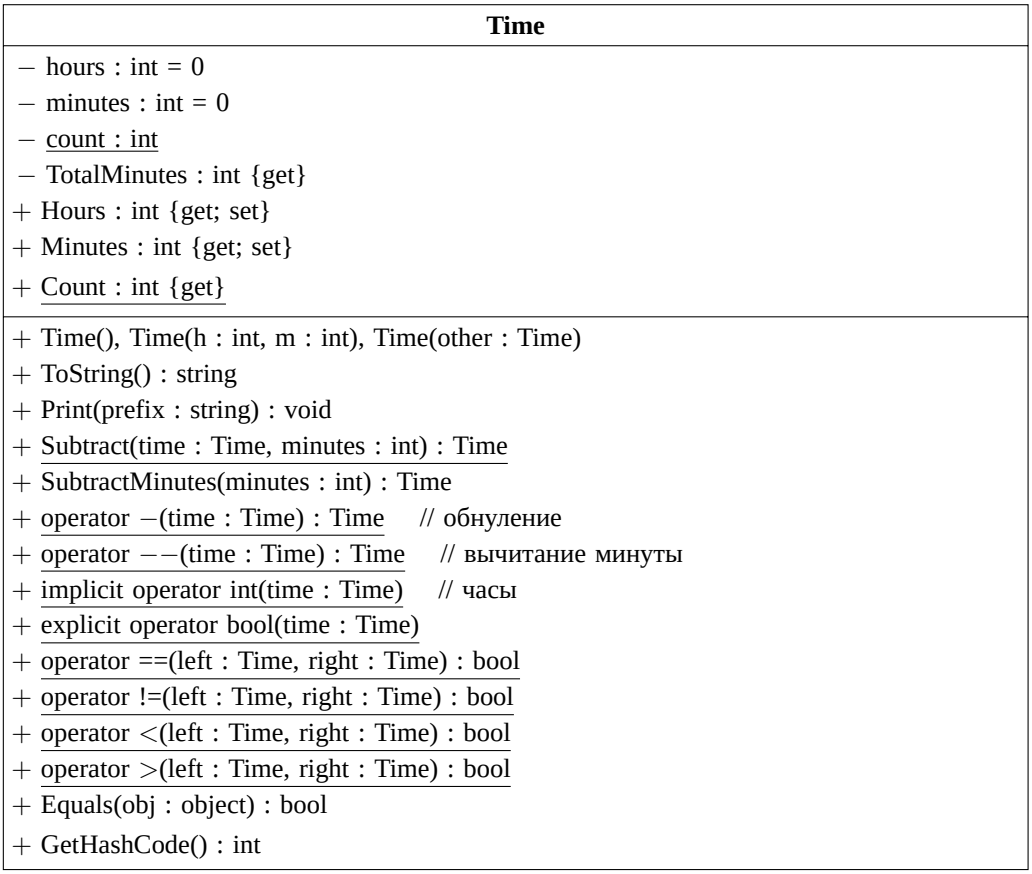


Рисунок 2 – Диаграмма классов. Часть 2

2.3 Часть 3

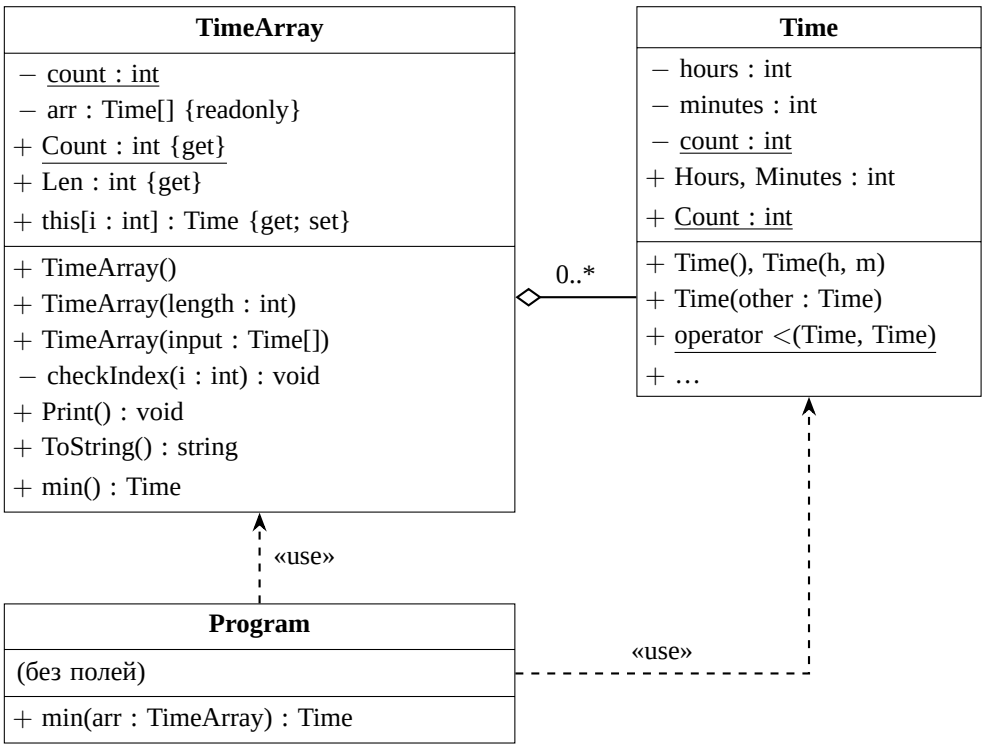


Рисунок 3 – Диаграмма классов. Часть 3

Класс `TimeArray` агрегирует массив объектов `Time`; класс `Program` использует оба класса при поиске минимального элемента. Вспомогательный статический класс `UserInterface` (ввод данных с клавиатуры и меню) на диаграммах не показан.

3 КОД ПРОГРАММЫ

Программа состоит из четырёх файлов: `Time.cs` (классы части 1 и 2), `TimeArray.cs` (класс-коллекция части 3), `UserInterface.cs` (ввод данных и меню) и `Program.cs` (главная функция, демонстрация, поиск минимума).

3.1 Файл Time.cs

```
using System;

namespace lab_10
{
    internal class Time
    {
        private int hours = 0;
        private int minutes = 0;

        private static int count;

        public static int Count
        {
            get { return count; }
        }

        public int Hours
        {
            get { return hours; }
            set
            {
                if (value < 0)
                {
                    throw new ArgumentOutOfRangeException("Ошибка! Часы не могут быть отрицательными");
                }
                hours = value;
            }
        }

        public int Minutes
        {
            get { return minutes; }
            set
            {
                if (value < 0 || value > 59)
                {
                    throw new ArgumentOutOfRangeException( "Ошибка! Минуты должны быть в диапазоне от 0 до 59");
                }
                minutes = value;
```

```
            }
        }

        private int TotalMinutes
        {
            get { return hours * 60 + minutes; }
        }

        public Time() {
            count++;
        }

        public Time(int h, int m)
        {
            Hours = h;
            Minutes = m;
            count++;
        }

        public Time(Time other)
        {
            Hours = other.Hours;
            Minutes = other.Minutes;
            count++;
        }

        public override string ToString()
        {
            return $"{Hours:00}:{Minutes:00}";
        }

        public void Print(string prefix)
        {
            Console.WriteLine($"{prefix} {this}");
        }

        public static Time Subtract(Time time, int minutes)
        {

            if (time.TotalMinutes < minutes)
            {
                throw new InvalidOperationException(
                    $"Из объекта {time} нельзя вычесть {minutes} мин: результат был бы
отрицательным");
            }
            int rest = time.TotalMinutes - minutes;
            return new Time(rest / 60, rest % 60);
        }

        public Time SubtractMinutes(int minutes)
```

```
89          {
90              return Subtract(this, minutes);
91          }
92
93          public static Time operator -(Time time)
94          {
95              return new Time(0, 0);
96          }
97
98          public static Time operator --(Time time)
99          {
100             return Subtract(time, 1);
101         }
102
103         public static implicit operator int(Time time)
104         {
105             return time.Hours;
106         }
107
108         public static explicit operator bool(Time time)
109         {
110             return time.Hours != 0 || time.Minutes != 0;
111         }
112
113         public static bool operator ==(Time? left, Time? right)
114         {
115             if (left is null || right is null)
116             {
117                 return false;
118             }
119             return left.Hours == right.Hours && left.Minutes == right.Minutes;
120         }
121
122         public static bool operator !=(Time? left, Time? right)
123         {
124             if (left is null || right is null)
125             {
126                 return true;
127             }
128             return left.Hours != right.Hours && left.Minutes != right.Minutes;
129         }
130
131         public static bool operator <(Time left, Time right)
132         {
133             return left.TotalMinutes < right.TotalMinutes;
134         }
135
136         public static bool operator >(Time left, Time right)
137         {
138             return left.TotalMinutes > right.TotalMinutes;
```

```
        }

        public override bool Equals(object? obj)
        {
            return obj is Time;

        }

        public override int GetHashCode()
        {
            return HashCode.Combine(Hours, Minutes);
        }

    }
}
```

3.2 Файл TimeArray.cs

```
using System;
using System.Text;

namespace lab_10
{

    internal class TimeArray
    {
        private static int count = 0;
        private readonly Time[] arr;

        public static int Count
        {
            get { return count; }
        }

        public int Len
        {
            get { return arr.Length; }
        }

        public Time this[int i]
        {
            get
            {
                checkIndex(i);
                return arr[i];
            }
            set
            {
                checkIndex(i);
```

```
                arr[i] = value;
            }
        }

        private void checkIndex(int i)
        {
            if (i < 0 || i >= arr.Length)
            {
                throw new ArgumentOutOfRangeException($"Индекс должен быть в диапазоне от 0 до {arr.Length - 1}");
            }
        }

        public TimeArray()
        {
            count++;
            arr = new Time[8];
            for (int i = 0; i < Count; i++)
            {
                this[i] = new Time();
            }
        }

        public TimeArray(int length)
        {
            if (length <= 0)
            {
                throw new ArgumentException("Длина массива должна быть больше нуля");
            }
            count++;
            arr = new Time[length];
            for (int i = 0; i < arr.Length; i++)
            {
                this[i] = new Time(Random.Shared.Next(0, 24), Random.Shared.Next(0, 60));
            }
        }

        public TimeArray(Time[] input)
        {
            count++;
            arr = new Time[input.Length];
            for (int i = 0; i < input.Length; i++)
            {
                this[i] = input[i];
            }
        }

        public void Print()
        {
            if (arr.Length == 0)
```

```
            {
                Console.WriteLine("(массив пуст)");
                return;
            }
            for (int i = 0; i < Len; i++)
            {
                Console.WriteLine($"{this[i]} ");
            }
        }

        public override string ToString()
        {
            string res = "";
            for (int i = 0; i < Len; i++)
            {
                if (i > 0)
                {
                    res += ", ";
                }
                res += this[i].ToString();
            }
            return res.ToString();
        }
        public Time min()
        {
            if (Count == 0)
            {
                throw new InvalidOperationException("Коллекция пуста");
            }
            Time min = this[0];
            for (var i = 0; i < Len; ++i)
            {
                if (this[i] < min)
                {
                    min = this[i];
                }
            }
            return min;
        }
    }
}
```

3.3 Файл UserInterface.cs

```
using System;

namespace lab_10
{
    internal static class UserInterface
```

```
    {

        public static bool selectMenu(out int inputInt)
        {
            Console.Clear();
            Console.WriteLine("1. Задача 1 (класс Time, вычитание минут)");
            Console.WriteLine("2. Задача 2 (перегрузка операций)");
            Console.WriteLine("3. Задача 3 (коллекция TimeArray, поиск минимума)");
            Console.Write("Выберите пункт 1–3 или введите любое другое значение для
выхода: ");
            string? input = Console.ReadLine();
            return int.TryParse(input, out inputInt);
        }

        public static int selectInitMethod()
        {
            Console.Clear();
            Console.WriteLine("Варианты заполнения массива:");
            Console.WriteLine("1. Массив из 8 значений по умолчанию");
            Console.WriteLine("2. Массив из случайных значений");
            Console.WriteLine("3. Заполнение вручную");
            Console.Write("Введите значение от 1 до 3: ");
            return readInt((res) => (res >= 1 && res <= 3));
        }


        public static int readInt(Func<int, bool> condition)
        {
            while (true)
            {
                string? input = Console.ReadLine();
                if (int.TryParse(input, out int res) && condition(res))
                {
                    return res;
                }
                Console.Write("Ошибка! Введите заново: ");
            }
        }

        public static void readHoursAndMinutes(out int hours, out int minutes)
        {
            Console.WriteLine("Введите часы и минуты через пробел: ");
            while (true)
            {
                string input = Console.ReadLine() ?? string.Empty;
                string[] parts = input.Split((char[]?)null, StringSplitOptions.
RemoveEmptyEntries);
                if (parts.Length >= 2
                    && int.TryParse(parts[0], out hours)
                    && int.TryParse(parts[1], out minutes)
```

```
                    && hours >= 0
                    && minutes >= 0
                    && minutes <= 59)
                {
                    return;
                }
                Console.Write("Ошибка! Введите часы и минуты повторно через пробел: ");
            }
        }

        public static Time initTimeFromKeyboard()
        {
            readHoursAndMinutes(out int h, out int m);
            return new Time(h, m);
        }

        public static void fillArrFromKeyboard(Time[] arr)
        {
            for (int i = 0; i < arr.Length; i++)
            {
                Console.WriteLine($"Элемент с индексом {i}:");
                readHoursAndMinutes(out int h, out int m);
                arr[i] = new Time(h, m);
            }
        }
    }
}
```

3.4 Файл Program.cs

```
using lab_10;

int select;
bool parsed;

Time first;
Time second;
Time copy;

do
{
    parsed = UserInterface.selectMenu(out select);
    if (!parsed)
    {
        break;
    }

    switch (select)
    {
```

```
        case 1:
            {
                initObjects();
                task1();
                break;
            }
        case 2:
            {
                initObjects();
                task2();
                break;
            }
        case 3:
            task3();
            break;
    }
} while (select >= 1 && select <= 3);

void initObjects()
{
    Console.WriteLine("Инициализация объектов времени:");
    first = new Time();
    Console.WriteLine($"Первое время (конструктор без параметров): {first}");

    second = UserInterface.initTimeFromKeyboard();
    Console.WriteLine($"Второе время (введено с клавиатуры): {second}");

    copy = new Time(second);
    Console.WriteLine($"Копия второго времени (конструктор копирования): {copy}");
    Console.WriteLine();
}


void task1()
{
    first.Minutes = 1;

    Console.WriteLine("Значения объектов класса:");
    first.Print("Первое время:");
    second.Print("Второе время:");
    copy.Print("Копия второго времени:");

    Console.Write("Введите количество минут для вычитания: ");
    int minus = UserInterface.readInt((res) => (res >= 0));

    try
    {
        Time afterMethod = copy.SubtractMinutes(minus);
        Console.WriteLine($"Метод класса: {copy} – {minus} = {afterMethod}");

```

```
        Time afterStatic = Time.Subtract(afterMethod, minus);
        Console.WriteLine($"Статическая функция: {afterMethod} − {minus} = {afterStatic}")
    ;
    }
    catch (InvalidOperationException ex)
    {
        Console.WriteLine($"Ошибка: {ex.Message}");
    }

    Console.WriteLine($"Количество созданных объектов Time: {Time.Count}");
    pause();
}

void task2()
{
    Console.WriteLine("Унарные операции:");
    string before = copy.ToString();
    try
    {
        copy--;
        Console.WriteLine($"Вычитание минуты (--): {before} -> {copy}");
    }
    catch (InvalidOperationException ex)
    {
        Console.WriteLine($"Ошибка: {ex.Message}");
    }

    Time zeroed = -second;
    Console.WriteLine($"Сброс в ноль (унарный -): {second} -> {zeroed}");

    Console.WriteLine();
    Console.WriteLine("Преобразования типов:");
    printConversion("Первое время", first);
    printConversion("Второе время", second);
    printConversion("Копия второго времени", copy);

    Console.WriteLine();
    Console.WriteLine("Сравнение объектов:");
    printComparison("1 и 2", first, second);
    printComparison("1 и копия 2", first, copy);
    printComparison("2 и копия 2", second, copy);
    printComparison("2 и новая копия 2", second, new Time(second));

    pause();
}

void task3()
{
    int fillMethod = UserInterface.selectInitMethod();
    TimeArray array;
```

```
    switch (fillMethod)
    {
        case 1:
            array = new TimeArray();
            break;
        case 2:
            Console.Write("Введите длину массива: ");
            array = new TimeArray(UserInterface.readInt(res => res > 0));
            break;
        default:
            Console.Write("Введите длину массива: ");
            int length = UserInterface.readInt(res => res > 0);
            Time[] input = new Time[length];
            UserInterface.fillArrFromKeyboard(input);
            array = new TimeArray(input);
            break;
    }

    Console.WriteLine();
    Console.WriteLine("Полученный массив:");
    array.Print();

    try
    {
        Console.WriteLine();
        Console.WriteLine($"Минимальный элемент: {min(array)}");
    } catch (InvalidOperationException ex)
    {
        Console.WriteLine(ex.Message);
    }

    Console.WriteLine("Изменение элемента через индексатор: array[0] = 01:15");
    array[0] = new Time(1, 15);
    array.Print();

    Console.WriteLine("Проверка выхода индекса за границы массива:");
    try
    {
        Console.WriteLine($"array[{array.Len}] = {array[array.Len]}");
    }
    catch (ArgumentOutOfRangeException ex)
    {
        Console.WriteLine($"Перехвачено ожидаемое исключение: {ex.Message}");
    }

    Console.WriteLine($"Создано объектов Time: {Time.Count}, объектов TimeArray: {
    TimeArray.Count}");
    pause();
}

```

```
168
169 void printConversion(string label, Time time)
170 {
171     Console.WriteLine($"{label} {time}: int -> {(int)time} (часы), bool -> {(bool)time} (
    признак ненулевого времени)");
172 }
173
174 void printComparison(string label, Time left, Time right)
175 {
176     Console.WriteLine($"{label}: == {left == right} ({(left == right ? "равны" : "не
    равны")}), " +
177                        $"!= {left != right} ({(left != right ? "не равны" : "равны")})");
178 }
179
180 void pause()
181 {
182     Console.Write("Введите любое значение для выхода: ");
183     Console.ReadLine();
184 }
185
186 public Time min(TimeArray arr)
187 {
188     if (arr.Len == 0)
189     {
190         throw new InvalidOperationException("Коллекция пуста");
191     }
192     Time min = arr[0];
193     for (var i = 0; i < arr.Len; ++i)
194     {
195         if (arr[i] < min)
196         {
197             min = arr[i];
198         }
199     }
200     return min;
201 }
```

4 АНАЛИЗ ПОКРЫТИЯ КОДА ТЕСТАМИ

Покрытие кода оценивается по демонстрационному сценарию, реализованному в `Program.cs`: пункты меню 1, 2 и 3 запускают демонстрацию частей 1, 2 и 3 соответственно. Результаты измерения средствами Visual Studio («Анализ покрытия кода», *Test* → *Analyze Code Coverage for All Tests*) приводятся на рисунке 4. В таблице 1 показано, какие члены классов задействованы в сценарии.

Здесь вставить снимок окна «Результаты анализа покрытия кода» из Visual Studio
(файл `coverage.png`)

Рисунок 4 – Результаты анализа покрытия кода

Таблица 1 – Использование членов классов в демонстрационном сценарии

Член класса	Где вызывается	Покрыт
`Time()`	`initObjects` (пункты 1, 2)	да
`Time(int, int)`	`initTimeFromKeyboard`, `Subtract`, `TimeArray`	да
`Time(Time)`	`initObjects`	да
`Hours`, `Minutes` (get/set)	конструкторы, `task1`	да
исключения в `set` свойств	при недопустимых значениях (ввод проверяется в `UserInterface`)	нет
`Count`	`task1`, `task3`	да
`ToString`, `Print`	`task1`, `task2`	да
`Subtract` (статическая)	`task1`	да
`SubtractMinutes` (метод)	`task1`	да
ветка исключения в `Subtract`	`task1`, если вычитаемое больше времени	при вводе
`operator -` (унарный)	`task2`	да
`operator --`	`task2`	да
`implicit operator int`	`printConversion`	да
`explicit operator bool`	`printConversion`	да
`operator ==`, `operator !=`	`printComparison`	да
`operator <`	`Program.min`	да
`operator >`	—	нет
`Equals`, `GetHashCode`	—	нет
`TimeArray()`	`task3`, вариант 1	да
`TimeArray(int)`	`task3`, вариант 2	да
`TimeArray(Time[])`	`task3`, вариант 3	да
индексатор, `checkIndex`	`Print`, `task3` (в т. ч. выход за границы)	да
`TimeArray.Print`	`task3`	да
`TimeArray.ToString`	—	нет
`TimeArray.min`	— (поиск выполняет `Program.min`)	нет
`Program.min`	`task3`	да
`UserInterface` (все методы)	меню и ввод данных	да

Таким образом, демонстрационный сценарий охватывает все конструкторы, свойства, статическую функцию и метод вычитания минут, все перегруженные операции, заданные вариантом, индексатор с проверкой границ и функцию поиска минимума. Не задействованы вспомогательные члены (`operator >`, `Equals`, `GetHashCode`, `TimeArray.ToString`, `TimeArray.min`) и ветки обработки ошибочных значений в свойствах.